Experiment No. 4

Aim :

Implement K-Nearest Neighbors (KNN) and evaluate model performance

Theory :

1. Dataset Source

Dataset Name: **Penguin dataset:EDA, classification and clustering**
Source Platform: Kaggle

Dataset Link :
https://www.kaggle.com/code/pratik1120/penguin-dataset-eda-classification-and-clustering/input

2. Dataset Description

The Palmer Penguins dataset consists of biological and physical measurements of penguins. The goal of the experiment is to classify penguin species using the K-Nearest Neighbors algorithm.

Features in the Dataset:

- **bill_length_mm** – Length of the penguin's bill
- **bill_depth_mm** – Depth of the bill
- **flipper_length_mm** – Flipper length measurement
- **body_mass_g** – Body mass of penguin
- **island** – Island where penguin lives
- **sex** – Gender of penguin

Target Variable:

- **species** – Penguin species (Adelie, Chinstrap, Gentoo)

Dataset Characteristics:

- Multiclass classification problem
- Numeric and categorical attributes
- Requires preprocessing such as encoding and scaling
- Some missing values that were removed during preprocessing

This dataset is suitable for KNN because the algorithm performs well on structured numeric datasets with clear feature relationships.

3. Mathematical Formulation of the Algorithm

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm used for classification and regression. It works by finding the K closest data points to a given sample based on distance measures.

The most commonly used distance metric is Euclidean Distance:

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Working Principle:

1. Calculate distance between the test sample and all training samples.
2. Select the K nearest neighbors.
3. Assign the class with the majority vote among neighbors.

Because KNN relies on distance calculation, feature scaling is necessary to avoid bias from large numeric values.

4. Algorithm Limitations

Although KNN is simple and effective, it has several limitations:

- **High Computational Cost:** Prediction becomes slow with large datasets since distance is calculated for all points.
- **Sensitive to Feature Scaling:** Without normalization, features with large values dominate distance calculation.
- **Curse of Dimensionality:** Performance decreases when the number of features increases significantly.
- **Memory Intensive:** Stores entire training dataset.
- **Sensitive to Noise:** Outliers can negatively affect classification results.

KNN may not perform well on very large datasets or high-dimensional data.

5. Methodology / Workflow

Step 1: Data Loading

The Palmer Penguins dataset was loaded into a Pandas DataFrame.

Step 2: Data Preprocessing

- Missing values were removed.
- Categorical features such as island and sex were encoded using Label Encoding.
- Features and target variables were separated.

Step 3: Data Visualization

Several visualizations were created including:

- Pairplot
- Correlation Heatmap
- Scatter Plot

- Feature Distribution Graphs

These helped understand relationships between features.

Step 4: Train-Test Split

The dataset was divided into training and testing sets using an 80:20 ratio.

Step 5: Feature Scaling

StandardScaler was applied to normalize feature values since KNN is distance-based.

Step 6: Model Training

A KNeighborsClassifier model was trained using training data.

Step 7: Model Evaluation

Predictions were made on test data and evaluated using accuracy score, confusion matrix, and classification report.

6. Performance Analysis

The performance of the KNN model was evaluated using classification metrics.

[13]
✓ 0s

```
print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.9850746268656716
[[30  1  0]
 [ 0 13  0]
 [ 0  0 23]]
              precision    recall  f1-score   support

           0       1.00      0.97      0.98        31
           1       0.93      1.00      0.96        13
           2       1.00      1.00      1.00        23

    accuracy                           0.99        67
   macro avg       0.98      0.99      0.98        67
weighted avg       0.99      0.99      0.99        67
```

7. Hyperparameter Tuning

Hyperparameter tuning was performed by varying the value of **K (number of neighbors)** from 1 to 20.

Steps followed:

1. Multiple KNN models were trained with different K values.
2. Accuracy was calculated for each value.
3. An Accuracy vs K graph was plotted.
4. The K value giving the highest accuracy was selected as the optimal parameter.

It was observed that very small K values caused overfitting, while larger K values reduced model flexibility. The optimal K provided the best balance between bias and variance.

Output :

[2]
4s

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

[5]

```python
df = sns.load_dataset("penguins")
print(df.head())
```

```
  species     island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0  Adelie  Torgersen            39.1           18.7              181.0
1  Adelie  Torgersen            39.5           17.4              186.0
2  Adelie  Torgersen            40.3           18.0              195.0
3  Adelie  Torgersen             NaN            NaN                NaN
4  Adelie  Torgersen            36.7           19.3              193.0

   body_mass_g     sex
0       3750.0    Male
1       3800.0  Female
2       3250.0  Female
3          NaN     NaN
4       3450.0  Female
```

[6]

```python
df = df.dropna()
```

[7]

```python
le = LabelEncoder()

df['species'] = le.fit_transform(df['species'])
df['island'] = le.fit_transform(df['island'])
df['sex'] = le.fit_transform(df['sex'])
```

[8]

```python
X = df.drop('species', axis=1)
y = df['species']
```

[9]

```python
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

[10]

```python
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

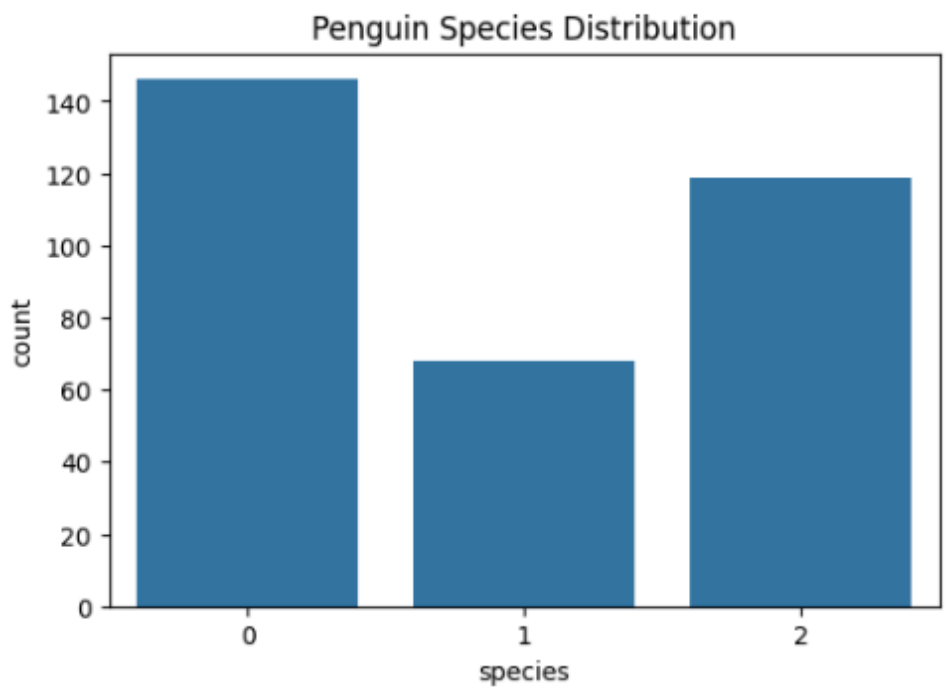
[11]
0s

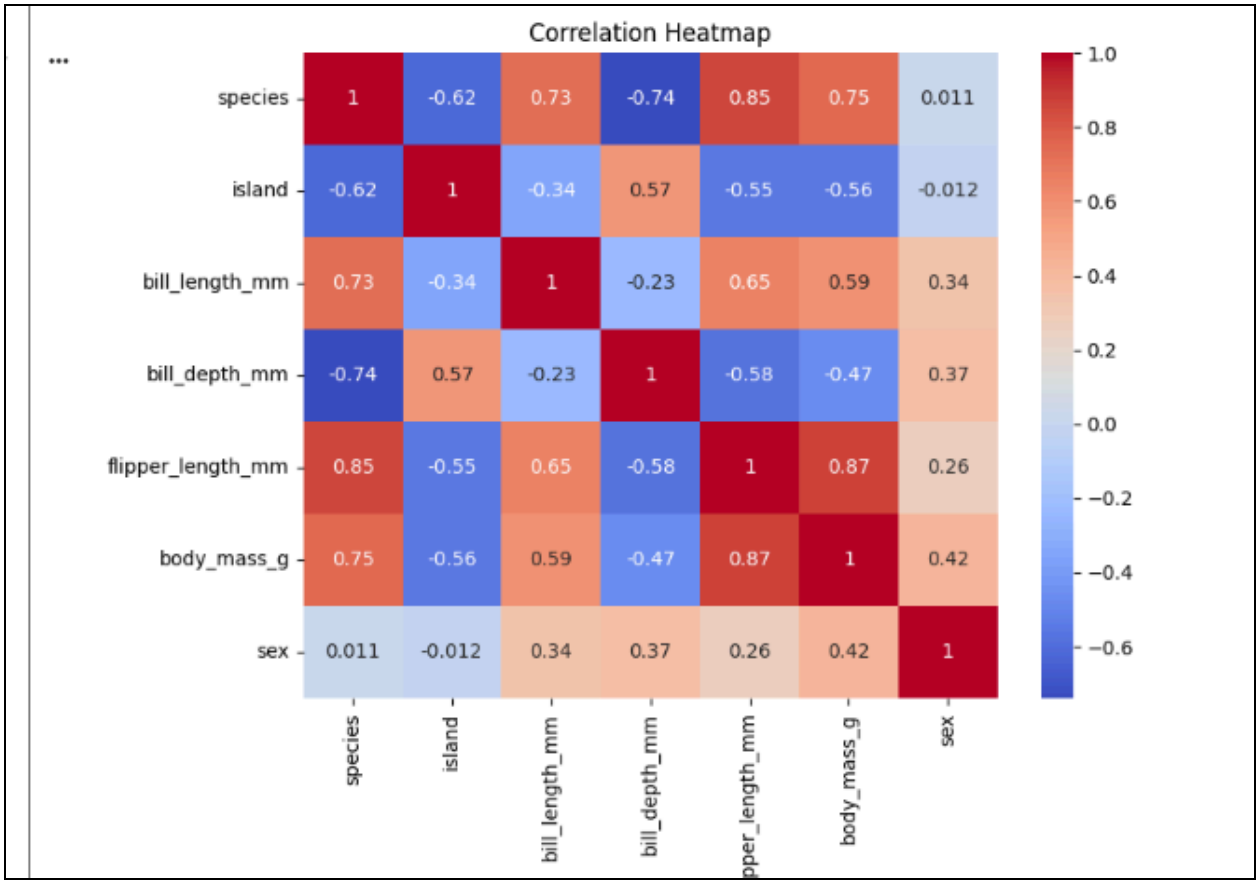
```
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
```

```
KNeighborsClassifier
KNeighborsClassifier(n_neighbors=3)
```

[18]
0s

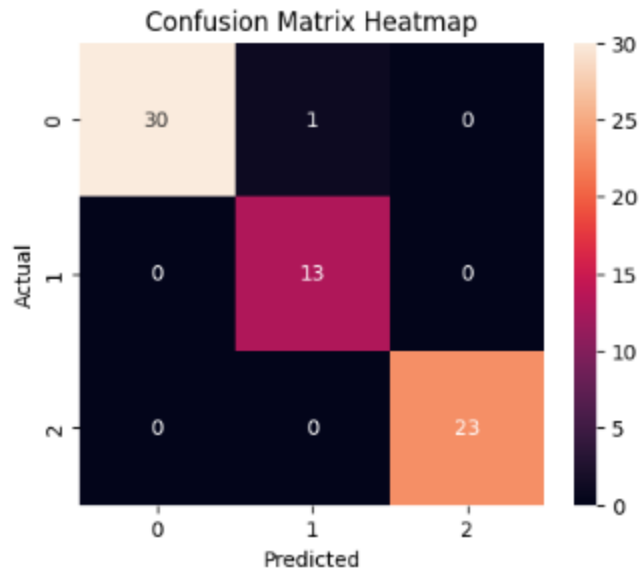
```
plt.figure(figsize=(6,4))
sns.countplot(data=df, x='species')
plt.title("Penguin Species Distribution")
plt.show()
```




Correlation Heatmap
species
island
bill_length_mm
bill_depth_mm
flipper_length_mm
body_mass_g
sex
1 -0.62 0.73 -0.74 0.85 0.75 0.011
-0.62 1 -0.34 0.57 -0.55 -0.56 -0.012
0.73 -0.34 1 -0.23 0.65 0.59 0.34
-0.74 0.57 -0.23 1 -0.58 -0.47 0.37
0.85 -0.55 0.65 -0.58 1 0.87 0.26
0.75 -0.56 0.59 -0.47 0.87 1 0.42
0.011 -0.012 0.34 0.37 0.26 0.42 1
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6

```
[22]
✓ 0s
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix Heatmap")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



Conclusion :

The K-Nearest Neighbors algorithm was successfully implemented using the Palmer Penguins dataset. After preprocessing, visualization, and hyperparameter tuning, the model achieved strong classification performance. The experiment demonstrated how distance-based learning can effectively classify real-world biological data.